Lab 4: VibeCoding

Project Report and Chat History: Endless Runner (Pygame)

Student (GitHub)	sravya-cloud
Assigned repo	SETAPESU26/39_endless-runner (forked to sravya-cloud/39_endless-runner)
Submission repo	https://github.com/sravya-cloud/39_endless-runner (folder: Lab-4)
Tool used	Claude Code in VS Code (Claude Sonnet 5.5)
Stack	Python 3.13, Pygame 2.6.1

1. Objective

Use an AI coding tool to fix and extend a partially working Pygame endless runner by prompting one task at a time. The README lists four tasks: refine collision detection (and the speed ramp), add a Game Over screen, add replay with a difficulty choice, and add sound effects.

2. Starting state (found before any edits)

The first prompt was read-only: Claude Code read the README and all game files, ran the game, and listed issues with file and line numbers. The main findings were:

Issue	Detail
Tunneling	Collision was checked only at an obstacle's position after it moved. Once the per-frame step exceeds about 55 px, an obstacle can skip the player entirely, and it still scored as cleared.
Uncapped speed	Speed rose forever (+0.003 per frame), so the game became unplayable long before tunneling.
Stale speed	Each obstacle moved with the previous frame's speed; per-obstacle speed was redundant.
Float truncation	pygame.Rect truncated float positions, adding up to 1 px of hitbox error.
No Game Over screen	Game over only printed the score to the console.

3. Work done, one prompt per task

Task	What was implemented	Commit
Task 1: Collision and speed	MAX_SPEED = 18 cap; swept collision using each obstacle's previous x (swept_rect); a single engine speed passed to move(); float positions with consistent rounding. Verified with a script forcing speed 500: the old check missed the hit, the swept check caught it.	7c99ca8
Task 2: Game Over screen	Dimmed overlay over the frozen game with the final score; console print removed; a 30-frame (about half a second) input delay so a held jump key cannot dismiss the screen.	e45c2ac
Task 3: Replay and difficulty	Menu on the Game Over screen: 1 Easy (speed 4.5, spawn every 95 frames), 2 Medium (6, 70), 3 Hard (8, 55); Esc or Q quits. reset() clears all state; MAX_SPEED still applies on every level.	d84acef
Task 4: Sound	game/sounds.py generates jump, score and game-over tones in code (math and array only, no audio files or new packages). Falls back to silent play if the mixer fails.	ef6a2fa

Task	What was implemented	Commit
Housekeeping	.gitignore for __pycache__ and *.pyc; Lab-4 folder with runnable code, both videos and this report.	f1e04b1, 91a0cbf

4. Verification and limits

Each task was tested headless with Pygame's dummy video and audio drivers, and the Lab-4 copy was run the same way. These tests confirmed behavior such as the replay reset, one game-over sound per crash, and no crash when the mixer fails. They could not confirm how the Game Over screen and menu look, or how the sounds actually sound. Those were checked by playing the game on a real machine and are shown in the after video.

5. Deliverables (Lab-4 folder)

before.mp4 (original game, about 10 seconds); after.mp4 (final game with all four tasks); the updated code (main.py, requirements.txt, game/); and this chat history PDF. The code also remains at the repository root, unchanged. The work was pushed to the personal fork only, and no pull request was raised to SETAPESU26.

The pages below contain the Claude Code session in full: the read-only analysis prompt and its report, then the prompts for each task with Claude Code's code edits, test output and summaries.

Part A: Read-only analysis (Prompt 0)

Prompt sent to Claude Code:

```
Read README.md, main.py, requirements.txt and everything in game/. Do NOT edit any files yet.
1. Install dependencies and run the game to confirm it launches.
2. Summarize how the game loop, player, obstacle spawning and speed ramp work.
3. List every bug or weakness you find related to collision detection and speed, with file
   and line numbers.
4. Check git config (user.name, user.email), whether a repo-level .claude/settings.json or
   local git config overrides attribution or identity, and whether Claude Code adds a
   Co-Authored-By line. Tell me what you find, and don't commit anything.
```

Claude Code's report:

```
I haven't edited or committed anything. Install and launch worked: pygame installed, and `main.py` ran headless (dummy video

## 2. How it works

- **Loop** ([main.py:22-38](39_endless-runner/main.py#L22-L38)): runs at 60 FPS. Each frame it clears the screen, forwards e
- **Player** ([player.py](39_endless-runner/game/player.py)): a 30×40 rect at x=80. A jump (Space, Up or W, only when on the
- **Obstacles** ([obstacle.py](39_endless-runner/game/obstacle.py)): 25×40 rects that spawn at x=800 and move left by `speed
- **Spawning** ([game_engine.py:49-52](39_endless-runner/game/game_engine.py#L49-L52)): one obstacle every 70 frames, a fixe
- **Speed ramp** ([game_engine.py:19-22,46](39_endless-runner/game/game_engine.py#L19-L22)): speed starts at 6 and gains 0.0
- **Scoring** ([game_engine.py:72-75](39_endless-runner/game/game_engine.py#L72-L75)): +1 when an obstacle's right edge pass
- **Game over** ([game_engine.py:67-70,91-94](39_endless-runner/game/game_engine.py#L67-L70)): setting `game_over` freezes `

## 3. Collision and speed issues

1. **Tunneling.** [game_engine.py:67-70](39_endless-runner/game/game_engine.py#L67-L70) checks collision only at the obstacl
2. **Uncapped speed.** [game_engine.py:22,46](39_endless-runner/game/game_engine.py#L22) has no ceiling (the code's own NOTE
3. **Stale speed.** In [game_engine.py:54-56](39_endless-runner/game/game_engine.py#L54-L56) `obstacle.move()` runs before `
4. **Float truncation in hitboxes.** [obstacle.py:19](39_endless-runner/game/obstacle.py#L19) and [player.py:31](39_endless-
5. **Frame-rate-coupled speed.** [game_engine.py:22,46](39_endless-runner/game/game_engine.py#L22) and the per-frame movemen
6. **Fixed spawn interval.** [game_engine.py:24,50](39_endless-runner/game/game_engine.py#L24) spaces obstacles by a fixed 7
7. **Smaller issues:**
   - `self.distance` ([:28,79](39_endless-runner/game/game_engine.py#L79)) is never used.
   - `jump()` is still called after game over ([:33-35](39_endless-runner/game/game_engine.py#L33-L35)), which is harmless b
   - `_game_over_logged` is an ad-hoc `getattr` flag ([:91](39_endless-runner/game/game_engine.py#L91)).

## 4. Git and attribution

- **Identity:** the effective `user.name` is `sravya-cloud` and `user.email` is `sitamutnuri@gmail.com`, both from the globa
- **Repo-level overrides:** the repo's `.git/config` has no `[user]` section, only the remote, the branch and `vscode-merge-
- **Claude Code settings:** there is no repo-level `.claude/` directory, and no `settings*.json` in `~/.claude` or `C:\Proje
- **Co-Authored-By:** yes, it would be added. This session's instructions tell me to end commit messages with `Co-Authored-B
- **Identity mismatch:** commits would be authored as `sravya-cloud <sitamutnuri@gmail.com>`, while the Claude account email

If you want, I can start on Task 1 next. I'd cap the speed and add a swept collision check, so an obstacle is tested against
```

Part B: Implementation session (Tasks 1 to 4, then commit)

Lines starting with > are the prompts sent to Claude Code. Lines starting with the bullet are Claude Code's actions and replies.

```
> Fix Task 1 from the README, using the issues you found. Make these changes:
  1. Cap the speed ramp at a sensible maximum, tunable via a constant, so the
  game stays playable in long runs.
  2. Make collision swept: test the player's rect against the full span each
  obstacle travelled this frame, so no obstacle can pass through at any speed.
  Do not score an obstacle that was never checked.
  3. Remove the stale per-obstacle speed. Use one engine speed, updated before
  obstacles move.
  4. Store positions as floats and convert to rects consistently, to avoid
  truncation errors.
     Keep the changes minimal and don't start on Tasks 2 to 4. Then write a
  small throwaway script that forces a very high speed and shows an obstacle
  hitting the player. Run it, show me the output, and delete the script
  afterwards. Finish with a summary of what changed and why.

● I'll start by looking at the README and the code.

  Searched for 2 patterns, read 4 files (ctrl+o to expand)

● Update(game\obstacle.py)
  └  Added 14 lines, removed 7 lines
       1  import pygame
       2
       3  class Obstacle:
       4 -    def __init__(self, x, ground_y, speed, width=25, height=40)
         -:
       5 -        self.x = x
       4 +    def __init__(self, x, ground_y, width=25, height=40):
       5 +        self.x = float(x)
       6 +        self.prev_x = self.x
       7          self.width = width
       8          self.height = height
       8 -        self.y = ground_y - height
       9 -        self.speed = speed
       9 +        self.y = float(ground_y - height)
      10          self.scored = False
      11
      12 -    def move(self):
      13 -        self.x -= self.speed
      12 +    def move(self, speed):
      13 +        self.prev_x = self.x
      14 +        self.x -= speed
      15
      16      def off_screen(self):
      17          return self.x + self.width < 0
      18
      19      def rect(self):
      19 -        return pygame.Rect(self.x, self.y, self.width, self.hei
         -ght)
      20 +        return pygame.Rect(round(self.x), round(self.y), self.w
         +idth, self.height)
      21 +
      22 +    def swept_rect(self):
      23 +        """Rect covering every position this obstacle occupied
         +during the last move()."""
      24 +        left = round(self.x)
      25 +        right = round(self.prev_x) + self.width
      26 +        return pygame.Rect(left, round(self.y), right - left, s
         +elf.height)

● Update(game\player.py)
  └  Added 2 lines, removed 2 lines
       2
       3  class Player:
       4      def __init__(self, x, ground_y, width=30, height=40):
       5 -        self.x = x
       5 +        self.x = float(x)
       6          self.ground_y = ground_y
       7          self.width = width
       8          self.height = height
       9 -        self.y = ground_y - height
       9 +        self.y = float(ground_y - height)
      10          self.vy = 0
```

```
     11          self.gravity = 0.8
     12          self.jump_strength = -15
```

● Update(game\player.py)
  └ Added 1 line, removed 1 line

```
     28              self.on_ground = True
     29
     30      def rect(self):
     31 -        return pygame.Rect(self.x, self.y, self.width, self.hei
        -ght)
     31 +        return pygame.Rect(round(self.x), round(self.y), self.w
        +idth, self.height)
```

● Update(game\game_engine.py)
  └ Added 1 line, removed 2 lines

```
     17          self.player = Player(80, self.ground_y)
     18
     19          self.speed = 6
     20 -        # NOTE: speed keeps climbing every frame with no ceilin
        -g. See
     21 -        # Task 1 in the README for why that eventually breaks c
        -ollision.
     20          self.speed_increase_per_frame = 0.003
     21 +        self.max_speed = MAX_SPEED
     22
     23          self.spawn_interval = 70  # frames between obstacle spa
         wns
     24          self._spawn_timer = 0
```

● Update(game\game_engine.py)
  └ Added 2 lines

```
      8  BROWN = (120, 80, 40)
      9  DARK_GREEN = (30, 100, 30)
     10
     11 +MAX_SPEED = 18  # px/frame ceiling for the speed ramp
     12 +
     13  class GameEngine:
     14      def __init__(self, width, height):
     15          self.width = width
```

● Update(game\game_engine.py)
  └ Added 7 lines, removed 14 lines

```
     44          if self.game_over:
     45              return
     46
     47 -        self.speed += self.speed_increase_per_frame
     47 +        self.speed = min(self.speed + self.speed_increase_per_f
        +rame, self.max_speed)
     48          self.player.update()
     49
     50          self._spawn_timer += 1
     51          if self._spawn_timer >= self.spawn_interval:
     52              self._spawn_timer = 0
     53 -            self.obstacles.append(Obstacle(self.width, self.gro
        -und_y, self.speed))
     53 +            self.obstacles.append(Obstacle(self.width, self.gro
        +und_y))
     54
     55          for obstacle in self.obstacles:
     56 -            obstacle.move()
     57 -            obstacle.speed = self.speed
     56 +            obstacle.move(self.speed)
     57
     59 -        # NOTE: collision is only checked against each obstacle
        -'s rect
     60 -        # *after* it has already moved for the frame. There's n
        -o check
     61 -        # for whether the obstacle's path crossed the player al
        -ong the
     62 -        # way. Since self.speed has no cap, it eventually grows
        - large
     63 -        # enough that an obstacle can cover more horizontal dis
        -tance
     64 -        # in one frame than the player's own width - meaning it
        - can be
     65 -        # entirely to the right of the player on one frame and
        -entirely
     66 -        # to the left on the next, skipping the player's hitbox
     67 -        # completely and never registering a hit. See Task 1.
```

```
58 +        # Swept test: the whole span each obstacle covered this
   + frame, so
59 +        # no speed can let one skip past the player.
60 +        player_rect = self.player.rect()
61          for obstacle in self.obstacles:
69 -            if obstacle.rect().colliderect(self.player.rect()):
62 +            if obstacle.swept_rect().colliderect(player_rect):
63                  self.game_over = True
64                  return
65
```

Ran 2 shell commands (ctrl+o to expand)

● Task 1 is fixed: the speed ramp is capped and obstacles can no longer skip past the player at any speed. I checked this with a throwaway script, which I've since deleted. Note that I had no earlier notes from a prior session, so I based the fixes on the problems described in the README and the code comments.

Script output

```
The script forced the speed to 500 px/frame, about 17 times the player's
width:
player x=80.0..110.0, obstacle start x=200.0, speed=500
after 1 frame: obstacle x=-300.0 (right edge -275.0) -> fully left of player
point-in-time rect collides (old logic): False
swept rect collides (new logic):         True
game_over: True | score: 0
The obstacle jumped from right of the player to far left in one frame. The old
per-frame rect check missed it, while the swept check caught it and ended the
game with no score awarded.

The script's cap check only tested a loop I wrote in the script, not the
engine's own update. I did not run the real game loop; the cap itself is a
one-line min() in update().

What changed

- Speed cap (game_engine.py (39_endless-runner/game/game_engine.py)): a new
  MAX_SPEED = 18 constant, used as self.max_speed. The speed ramp now stops at
  that value, so long runs stay playable. 18 is my guess at a sensible
  ceiling, so adjust it if it feels too fast or too slow.
- Swept collision (obstacle.py (39_endless-runner/game/obstacle.py),
  game_engine.py (39_endless-runner/game/game_engine.py)):
  - Each obstacle now remembers where it was before it moved (prev_x). A new
    swept_rect() covers everything it passed through this frame, and the
    engine checks the player against that.
  - Collision runs before scoring and returns early on a hit, so an obstacle
    can only score after it has been checked. I changed no scoring code.
- One speed (all three files): obstacles no longer store their own speed. They
  now have move(speed), and the engine updates its single speed before moving
  them.
- Float positions (player.py (39_endless-runner/game/player.py), obstacle.py
  (39_endless-runner/game/obstacle.py)): positions are stored as floats.
  rect() and swept_rect() use round(), because pygame's Rect otherwise cuts
  off the decimals. That cut-off was dropping part of a pixel every frame.

I didn't touch Tasks 2 to 4.
```

> Implement Task 2 from the README. When the player collides with an obstacle, show a Game Over screen in the Pygame window with the final score, then wait for keyboard input. Remove the console print of the final score. Draw the screen over the frozen game or on a dimmed background, and use a font so the text stays readable. Show a prompt line such as "Press any key", but do not add replay or difficulty logic yet. Don't touch the Task 1 changes. Run the game headless to confirm it doesn't crash, and tell me what you changed.
Read 1 file, ran 2 shell commands (ctrl+o to expand)

● Task 2 is in: a collision now freezes the game and draws a Game Over screen with the final score, and the console print is gone.

I ran the real game loop headless with the dummy video driver, and it didn't crash. The run reached a collision with a score of 0 (the player never jumped). I didn't look at the rendered screen, so the layout and font readability are unchecked. I deleted the test script and the screenshot it saved.

What changed

- Screen (game_engine.py (39_endless-runner/game/game_engine.py)):

  - The frozen game stays visible under a semi-transparent black overlay.
  - It shows "Game Over" in a 64pt bold Arial, "Final Score: N", and "Press any key to exit" in the existing 30pt font. All text is white or light grey on the dark overlay.
  - The prompt line only appears once input is accepted.
- Input:
  - After the game ends, any key press sets should_quit.
  - Jump keys no longer do anything at that point.
  - main.py (39_endless-runner/main.py) checks should_quit and ends the loop, so pressing a key closes the window. That is the only action for now, since you didn't want replay logic.
- Mash guard: key presses in the first 30 frames (about half a second) after the crash are ignored. Otherwise the jump key you were hammering would close the screen before you could read it. Change GAME_OVER_INPUT_DELAY if you want a different delay. I checked that a key at frame 10 is ignored and one at frame 40 quits.
- Console print: removed, along with the _game_over_logged flag it used.

The Task 1 changes are untouched.

> Implement Task 3 from the README. On the Game Over screen, replace "Press any key to exit" with a menu: the player picks Easy, Medium or Hard to play again (keys 1, 2 and 3, with the labels shown on screen), or quits (Esc or Q). Each difficulty sets a different starting speed and obstacle spawn interval. Medium should match the current game, Easy should be slower with more space between obstacles, and Hard should be faster with less space. Keep the existing half-second input delay. Make replay reset all state cleanly (score, speed, obstacles, player position and velocity, spawn timer), so nothing carries over from the previous run. Keep MAX_SPEED capped on every difficulty, and check the Hard start speed stays below it. Don't touch Task 4. Run the engine headless through two full replays at different difficulties and tell me what you checked. Then give me a summary of what changed.

Read 1 file, ran 1 shell command (ctrl+o to expand)

● Task 3 is in: the Game Over screen now has a difficulty menu for replaying, and Esc or Q quits. I ran the engine headless through two full replays (Medium to Hard, then Hard to Easy), and nothing crashed. I did not view the rendered menu, so I haven't checked how it looks.

What I checked
- Start speeds vs. cap: Easy 4.5, Medium 6 and Hard 8 are all well below MAX_SPEED (18), which still applies on every difficulty.
- Replay 1 (Medium to Hard):
  - After a crash I pressed 3 at frame 5, and it was ignored.
  - I pressed 3 again at frame 30, and it started a new run.
  - The new run had score 0, speed 8, spawn interval 55, spawn timer 0, no obstacles and distance 0. The game-over flag and counter were reset, and the player was back at x=80, on the ground, with zero velocity.
  - It then played 300 more frames without trouble.
- Replay 2 (Hard to Easy): The same checks passed at speed 4.5 and spawn interval 95.
- Menu keys: Space on the Game Over screen did nothing, and both Q and Esc set the quit flag.
- Medium: it starts at speed 6 and spawn interval 70, the same as the original game.
- I deleted the test script afterwards.

What changed

All changes are in game_engine.py (39_endless-runner/game/game_engine.py).
- Difficulties: a new DIFFICULTIES table gives each level a key, starting speed and spawn interval:

Difficulty	Key	Starting speed	Spawn interval (frames)
Easy	1	4.5	95
Medium	2	6	70
Hard	3	8	55

  The Easy and Hard numbers are my picks, so tune them if you like.
- Clean reset: a new reset(difficulty) method rebuilds the player and clears the score, speed, spawn interval, spawn timer, obstacles, distance and game-over state. __init__ calls it with Medium. The fonts and the speed cap are set once, so a replay doesn't reload them.
- Input: on the Game Over screen, keys 1, 2 and 3 start a new run, and Esc or Q quits. All of it is gated by the existing 30-frame delay, and any other key is ignored.
- Menu: it replaces "Press any key to exit" with "Play again", the line "1: Easy 2: Medium 3: Hard", and "Esc / Q: Quit". It appears once input is

```
  accepted, as the old prompt did.

  The Task 1 changes and the main.py quit handling are untouched, and I didn't
  start Task 4.

> Implement Task 4 from the README. Add sound effects for jumping, scoring
  (when an obstacle is cleared) and the game-over moment. Requirements:
  1. Generate the sounds in code at startup, with no downloaded or bundled
  audio files. Use only the Python standard library (math, array) with
  pygame.mixer, so requirements.txt doesn't need a new package. If you do need
  numpy, tell me first and add it to requirements.txt.
  2. Keep them short and distinct: a quick rising blip for jump, a short high
  ping for score, and a longer falling tone for game over. Keep the volume
  moderate.
  3. Play the game-over sound once per crash only, not every frame, and don't
  replay it while the Game Over screen waits. Play the jump sound only when a
  jump actually starts, not on every key press in mid-air.
  4. If pygame.mixer fails to initialize or sound creation raises, the game
  must still run silently with no crash and no spam in the console.
  5. Make sure replay still works with sound, and that no sound plays during
  the Game Over delay or menu.
     Don't change Tasks 1 to 3. Run the engine headless with the dummy audio
  driver and confirm: the sounds are created, each event triggers its sound
  exactly once, and forcing a mixer failure leaves the game running. I can't
  hear headless output, so say clearly what you could not verify. Delete any
  test scripts afterwards and give me a summary of what changed.

● Write(game\sounds.py)
  └  Wrote 48 lines to game\sounds.py
       1 import array
       2 import math
       3
       5
       6 VOLUME = 0.4  # moderate overall level, 0.0 - 1.0
       7
       8
          volume=VOLUME):
      10      """Build a 16-bit sine sweep from freq_start to freq_end wit
          h a fade-out."""
     … +45 lines (ctrl+o to expand)

  Searched for 1 pattern, ran 2 shell commands (ctrl+o to expand)

● Task 4 is in: jump, score and game-over sounds are generated in code at
  startup. I ran it headless with the dummy audio driver and the checks passed,
  but I couldn't hear anything, so how the sounds actually sound is unverified.
  No numpy was needed, so requirements.txt is unchanged. I deleted the test
  script, and the __pycache__ folder my test runs created.

  What I checked

  - Sounds are created. The mixer came up at 44100 Hz, 16-bit, stereo, and all
    three sounds were built: jump 0.12 s, score 0.10 s, game over 0.70 s.
  - Jump. Three jump key presses in a row (the second two mid-air) played one
    jump sound.
  - Game over. The crash played the sound once. After 120 more frames of
    waiting, plus a space press and another stray key on the menu, nothing else
    played.
  - Replay. Pressing 3 to restart played no sound. A second crash on Hard played
    the game-over sound exactly once.
  - Score. A cleared obstacle played the score sound once and the score went
    to 1. I set up this obstacle by hand rather than playing through a real
    jump.
  - Mixer failures. I forced mixer.init to raise, then separately forced sound
    creation to raise. Both times the game ran 600 frames and handled a replay
    key press with no crash. The console printed nothing, and the sound bank was
    left empty. For these runs I didn't directly check that the game reached
    the Game Over screen, so the check was only that nothing crashed.

  For the "plays once" checks I swapped in stand-in sound objects that count
  calls, so these confirm when play is called, not that audio came out.
  What I could not verify

  - Whether the three sounds are distinct and pleasant. I chose the pitches and
    volume by design only.
  - Whether the volume of 0.4 is right on a real speaker.
  - Real hardware, a missing audio device and an audio driver that isn't 16-bit.
    I only forced the failures by patching.
  What changed
```

```
  - New sounds.py (39_endless-runner/game/sounds.py). It builds each sound from
    a sine sweep with a short fade-in and fade-out, using only math and array:
    - Jump: 380 to 760 Hz in 0.12 s.
    - Score: a 1320 Hz ping in 0.10 s.
    - Game over: 440 to 110 Hz in 0.7 s.
  - Failure handling. If the mixer or sound creation fails for any reason, the
    sound bank is left empty and play() does nothing, silently. Output formats
    other than 16-bit are also skipped.
  - game_engine.py (39_endless-runner/game/game_engine.py). It creates the sound
    bank once and plays sounds in three places:
    - The jump key plays a sound only when the player is on the ground. I
      checked on_ground in the engine rather than changing player.py.
    - The sound for a cleared obstacle plays right where the score goes up.
    - The game-over sound plays at the moment of collision. That branch returns
      straight away, so it can't repeat while the screen waits.

  The Task 1 to 3 logic is untouched.

> Finish the lab. Do not push yet.
  1. Add a .gitignore containing __pycache__/ and *.pyc. If any __pycache__
  files are tracked, remove them from tracking.
  2. Create a Lab-4 folder at the repo root. Copy into it: the final main.py,
  requirements.txt and game/ folder (so the folder is runnable on its own),
  plus the files C:\Projects\Videos\before.mp4 and
  C:\Projects\Videos\after.mp4. Leave a placeholder file
  Lab-4/chat-history-NOTE.txt saying the chat history PDF will be added. Keep
  the code at the repo root as well, unchanged.
  3. Run python main.py headless (dummy video and audio drivers) from inside
  Lab-4 for a few seconds to confirm the copy runs.
  4. Show me git status and the list of files to be committed. Flag any file
  over 50 MB.
  5. Commit in logical steps with clear messages: one for Task 1, one for Task
  2, one for Task 3, one for Task 4, one for the Lab-4 folder. If that's
  impractical, use one commit per task group and say so. Do not add any
  Co-Authored-By or "Generated with Claude Code" text to any commit message.
  Use the repo's existing git identity (sravya-cloud, noreply email) and don't
  change it.
  6. Run git log --format=full -n 10 and show it. Confirm that the author and
  committer are only sravya-cloud with the noreply address, and that no message
  mentions Claude.
  7. Run git remote -v and confirm origin is
  https://github.com/sravya-cloud/39_endless-runner.git and not SETAPESU26.
  Stop there and wait for me to say go before pushing.
  Ran 5 shell commands (ctrl+o to expand)

● Lab-4 runs headless for 6 seconds with no errors. I'm now committing it and
  then running the log, identity and remote checks.

  Ran 1 shell command (ctrl+o to expand)

● Everything is committed locally and nothing has been pushed. I'm waiting for
  your go.

  Commits

  There are six commits. One more than you asked for: I put the .gitignore and
  untracking in their own commit.
  91a0cbf Add Lab-4 submission folder
  ef6a2fa Task 4: add synthesised sound effects
  d84acef Task 3: add replay with Easy/Medium/Hard difficulty menu
  e45c2ac Task 2: add Game Over screen
  7c99ca8 Task 1: cap speed ramp and use swept collision
  f1e04b1 Add .gitignore and stop tracking __pycache__ files
  - Splitting by task: the task changes overlap in game_engine.py, so I rebuilt
    each in-between version of that file. Each commit holds only its own task's
    changes, and I ran the game headless at each of the four task commits
    without a crash. The final working tree is identical to the Task 4 content,
    with nothing left uncommitted.
  - .gitignore and __pycache__: the .gitignore holds __pycache__/ and *.pyc.
    Three tracked .pyc files were removed from tracking.
  - Lab-4 contents: the folder has main.py, requirements.txt, game/, both videos
    and the chat-history placeholder. The root code is unchanged.
  - Lab-4 run: I ran python main.py from inside Lab-4 headless for 6 seconds. It
    ran with no errors and was ended by my timeout (exit code 124), which is
    expected for a game that never stops.

  Check on your video file names

  The source files were actually named before.mp4.mp4 and after.mp4.mp4. I
```

copied them into Lab-4 as before.mp4 and after.mp4. The originals in C:\Projects\Videos are untouched.

Files and size

- Lab-4/ holds nine new files, all committed. The two videos are 20 MB (after) and 11 MB (before).
- No file is over 50 MB, and git status is clean.

Identity

- My six commits: Author and Committer are sravya-cloud <sravya-cloud@users.noreply.github.com>. I used the repo-local git config and didn't change it.
- Older commits: the two earlier commits are not mine. They are by Samrudha Narendra and kwazzy-coder, with committers GitHub and kwazzy-coder. They came with the repo and are not rewritten.
- Messages: none mention Claude, Co-Authored-By or "Generated with".

Remote

origin is https://github.com/sravya-cloud/39_endless-runner.git for both fetch and push, not SETAPESU26.

One thing for later

Your global git config has a different email, sitamutnuri@gmail.com. It doesn't affect this repo, because the repo's own setting takes priority, but it will apply in your other repos.